Packages

Java packages are a way to organize classes and interfaces into namespaces, providing a structured way to manage large codebases. Packages help in avoiding name conflicts, controlling access, and making the code more modular and maintainable.

1. What are Packages?

A package in Java is a namespace that organizes a set of related classes and interfaces. Think of it as a folder in a file directory. By using packages, you can group related classes together, making your code more organized and modular.

Example:

```
package com.example.myapp;



public class MyClass {

    // Class implementation

}
```

2. Benefits of Using Packages

- **Namespace Management**: Avoids naming conflicts by organizing classes into separate namespaces.
- **Access Control**: Helps in controlling the access to classes and interfaces.
- **Modularity**: Promotes modular design by grouping related classes together.
- **Code Reusability**: Makes it easier to reuse classes across different projects.
- **Organization**: Improves the organization of large codebases, making it easier to navigate and manage.

3. Creating and Using Packages

To create a package, use the `package` keyword at the beginning of your Java file, followed by the package name. The directory structure should match the package name.

Example:

Directory Structure:

```
src/
  com/
    example/
```

```
myapp/
    MyClass.java
```

MyClass.java:

```
package com.example.myapp;



public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in package com.example.myapp");
    }
}
```

4. Importing Packages

To use classes from a package, you need to import them using the `import` keyword. You can import a single class or all classes in a package using the wildcard `*`.

Example:

Using a Single Class:

```
import com.example.myapp.MyClass;


public class Main {
    public static void main(String[] args) {
        MyClass myClass = new MyClass();
        myClass.display();
    }
}
```

Using All Classes in a Package:

```
import com.example.myapp.*;


public class Main {
    public static void main(String[] args) {
        MyClass myClass = new MyClass();
        myClass.display();
```

```
    }

}
```

5. Built-in Packages in Java

Java comes with a rich set of built-in packages that provide various functionalities. Some commonly used built-in packages include:

- `java.lang`: Contains fundamental classes such as `String`, `System`, `Math`, etc.
- `java.util`: Contains utility classes such as `ArrayList`, `HashMap`, `Date`, etc.
- `java.io`: Contains classes for input and output operations.
- `java.nio`: Contains classes for non-blocking I/O operations.
- `java.net`: Contains classes for networking operations.
- `java.sql`: Contains classes for database access.

Example:

Using a Built-in Package:

```
import java.util.ArrayList;
```

```
public class Main {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Hello");

        list.add("World");

        System.out.println(list);

    }

}
```

Output:

```
[Hello, World]
```

6. Access Modifiers and Packages

Access modifiers in Java control the visibility of classes, methods, and fields. They play a crucial role in how packages are used.

- **Public**: The class, method, or field is accessible from any other class.

- **Protected**: The class, method, or field is accessible within the same package and subclasses.
- **Default (no modifier)**: The class, method, or field is accessible only within the same package.
- **Private**: The class, method, or field is accessible only within the same class.

Example:

```
package com.example.myapp;


public class MyClass {
    public void publicMethod() {
        System.out.println("Public method");
    }

    protected void protectedMethod() {
        System.out.println("Protected method");
    }
```

```
    void defaultMethod() {
        System.out.println("Default method");
    }

    private void privateMethod() {
        System.out.println("Private method");
    }
}
```

7. Package Naming Conventions

Java package names typically use the reverse domain name convention to avoid conflicts. The naming convention starts with the top-level domain, followed by the domain name and any subdomains or specific project names.

Example:

- `com.example.myapp`
- `org.company.project`

8. Example: Creating and Using Packages

Example: Creating a Package

Directory Structure:

```
src/
  com/
    example/
      myapp/
        MyClass.java
        AnotherClass.java
```

MyClass.java:

```
package com.example.myapp;


public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in package com.example.myapp");
```

```
    }

}
```

AnotherClass.java:

```
package com.example.myapp;



public class AnotherClass {

    public void show() {

        System.out.println("Hello from AnotherClass in
package com.example.myapp");

    }

}
```

Example: Using the Package

Main.java:

```
import com.example.myapp.MyClass;

import com.example.myapp.AnotherClass;
```

```
public class Main {

    public static void main(String[] args) {

        MyClass myClass = new MyClass();

        myClass.display();



        AnotherClass anotherClass = new AnotherClass();

        anotherClass.show();

    }

}
```

Output:

```
Hello from MyClass in package com.example.myapp

Hello from AnotherClass in package com.example.myapp
```

9. Real-World Analogy

Consider a library system:

- Books (classes) are organized into sections (packages) based on their subjects.

- Each section helps in locating and categorizing books, just as packages help in organizing and managing classes.